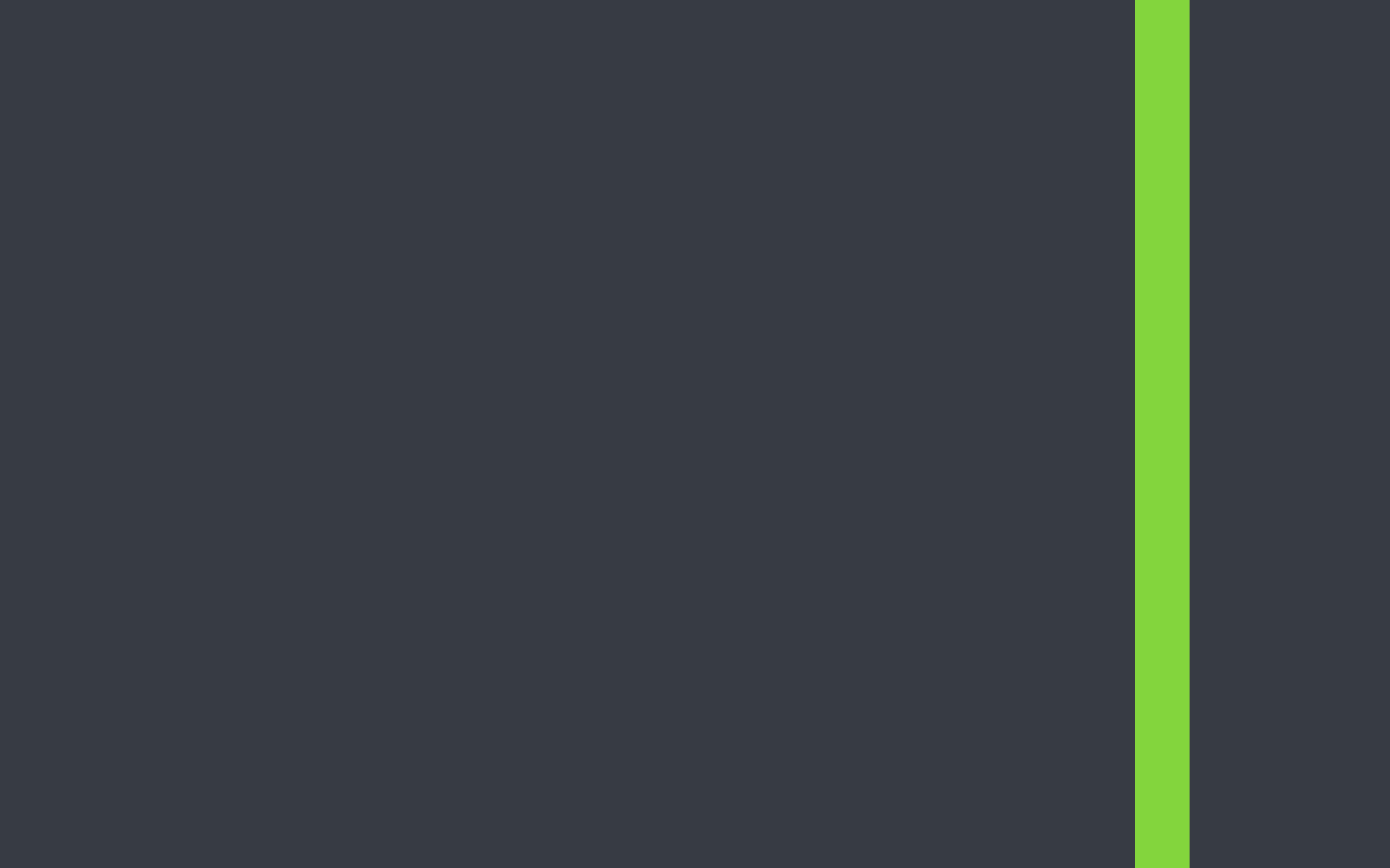




Katharà

Mattia Cosimi

December 2024

1 Introduzione

Comandi utili: **v-prefixed** commands si utilizzano su un singolo dispositivo: **vstart** avvia un nuovo dispositivo, **vconfig** Collega le interfacce di rete a un dispositivo in esecuzione, **vclean** arresta un dispositivo.

l-prefixed commands si utilizzano su un intero lab. **lstart** avvia un lab Katharà, **lclean** controlla l'interfaccia network di un device su un lab in esecuzione, **lrestart** arresta tutti i dispositivi del lab e li riavvia, **linfo** restituisce le informazioni del lab.

Ci sono poi i comandi globali: **check** verifica l'ambiente del sistema. **connect** connette a una macchina Katharà in esecuzione, **list** fa vedere tutte le macchine katharà in esecuzione del utente attuale, **settings** mostra e modifica le impostazioni Katharà, **wipe** elimina tutto.

■ To test if your setup works correctly, run:

- **kathara check**
 - this command runs automatic tests in your environment
- **kathara vstart -n pc1 --eth 0:A**

Figure 1

In un lab Katharà solitamente c'è un **lab.conf** che descrive la topologia di rete e i dispositivi che vengono avviati, una serie di **sottocartelle** dove solitamente sono situate le impostazioni di configurazione per ogni dispositivo e i file **.startup** nei quali vengono scritte le azioni che esegue un dispositivo quando viene avviato.

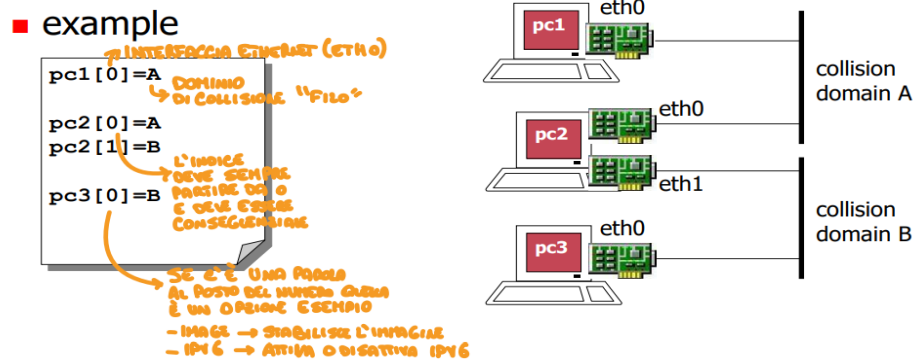


Figure 2

2 Primo Lab: due computer

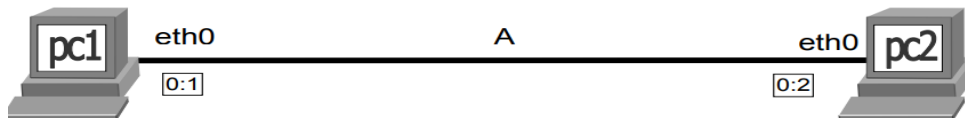


Figure 3

Nel lab.conf notiamo come pc1 ha un'interfaccia (eth0) connessa a un dominio di collisione A. In questo lab l'indirizzo MAC è specificato, se non lo fosse, Katharà gliene assegnerebbe uno **random**. Per cambiare un indirizzo MAC:

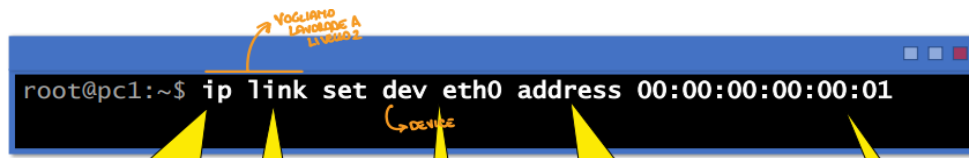


Figure 4

Vogliamo far comunicare i due computer e lo facciamo con frame Ethernet (Frame di livello 2) a tal proposito utilizziamo scapy, basta digitare scapy nel dispositivo, mentre per uscirci exit(). Creiamo il pacchetto:

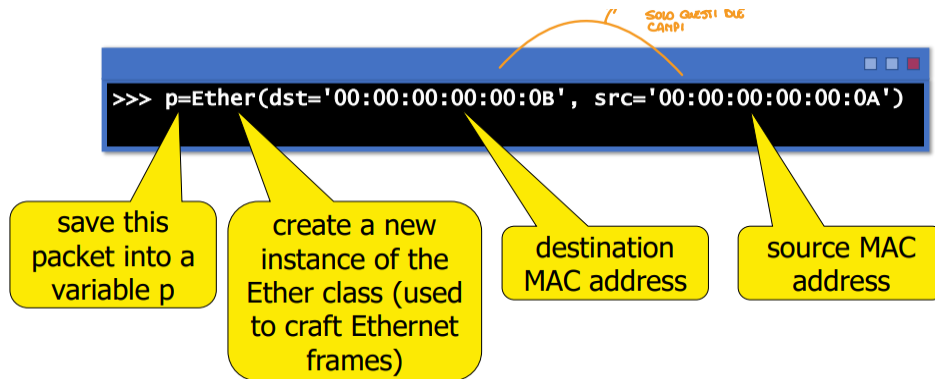


Figure 5

Inviando il pacchetto:

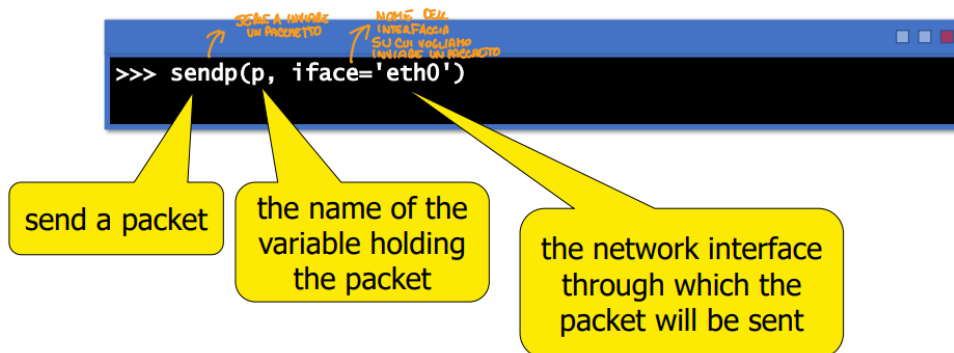


Figure 6

Per sapere se il pacchetto è stato ricevuto utilizziamo un **packet sniffer**, è uno strumento che cattura il traffico di rete che transita su un segmento di rete. A tal proposito utilizziamo **wireshark**. Per aprirlo basta digitare sulla barra di ricerca del browser <http://localhost:3000>. Ovviamente per analizzare il dominio di collisione bisogna **connettere** la macchina wireshark:

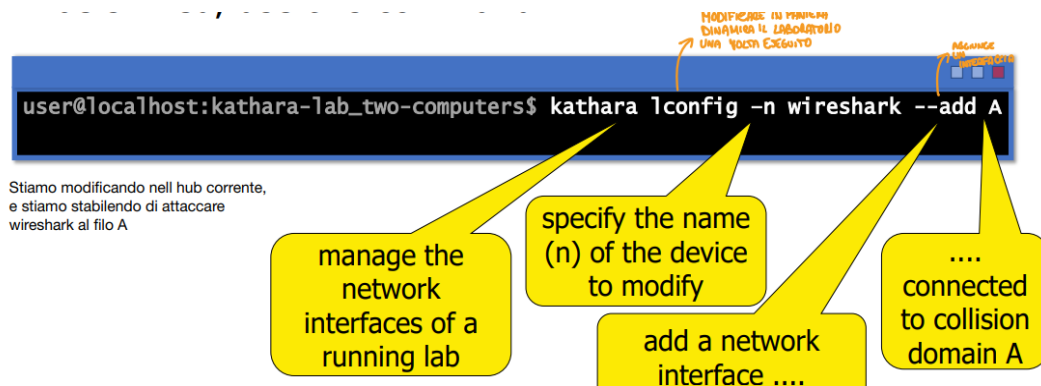


Figure 7

NOTA: quando connettiamo wireshark a un dominio di collisione per essere sniffato, viene aggiunta un interfaccia. La dovremo aprire per analizzare il traffico di rete.

3 Secondo Lab: un bridge

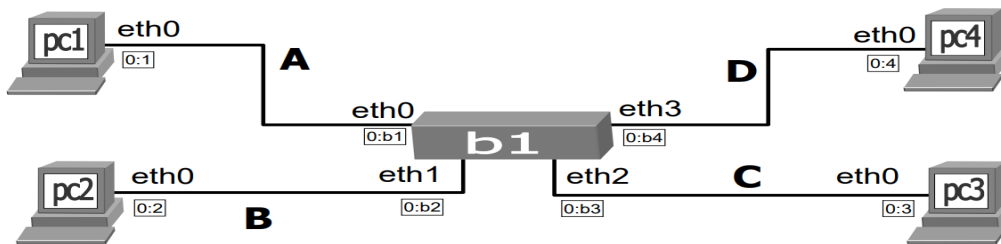


Figure 8

Per prima cosa creiamo un bridge:

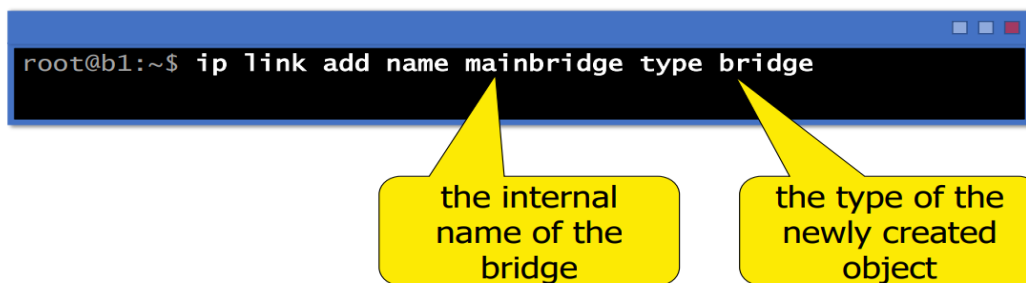


Figure 9

Quindi andiamo a creare un oggetto bridge all'interno di b1. Successivamente connettiamo l'interfaccia eth0:

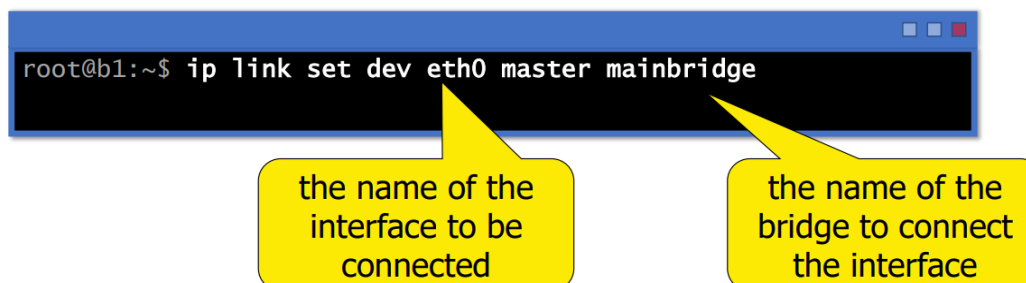


Figure 10

Quando un bridge viene creato il suo stato iniziale è DOWN. Andiamo quindi a settare il suo stato su up:



Figure 11

I dati del filtering database non rimangono per sempre, di default dopo 300 s vengono tolti dal filtering database, possiamo cambiare il tempo ("ageing time") così:



Figure 12

Tutti questi comandi sono già presenti sul file di startup di b1. Andiamo ora ad analizzare il filtering database del bridge:

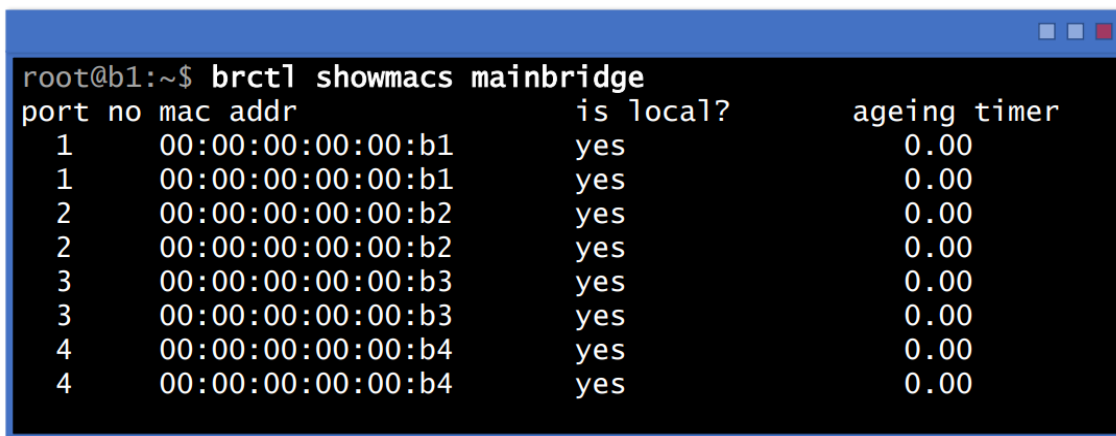


Figure 13

Il bridge di Linux impara automaticamente i MAC address delle sue interfacce locali. L'ageing time è a 0 dato che le interfacce locali non scadono mai. Port indica il numero di porte virtuali del bridge. L'ageing time indica quanto questo indirizzo è rimasto nel filtering database.

Successivamente inviamo un pacchetto con scapy e ci accorgiamo che il filtering database impara il MAC addr del mittente.

4 Terzo Lab: Configurazione di base IPv4, Ping, Traceroute e Arp

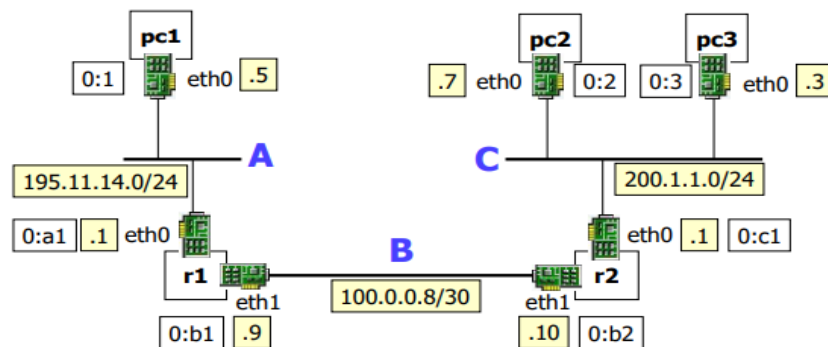


Figure 14

Nel lab.conf assegniamo a ogni host, un indirizzo IPv4 alle loro interfacce eth0. Inoltre aggiungiamo per ogni host, un default gateway.

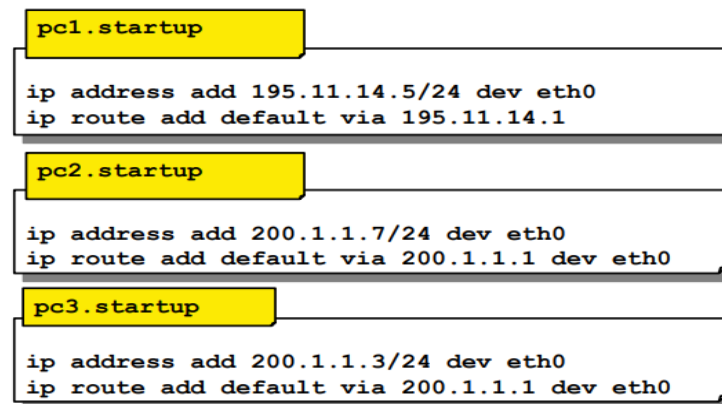


Figure 15

Configuriamo anche i router, a cui aggiungiamo i collegamenti alle macchine e il next hop.

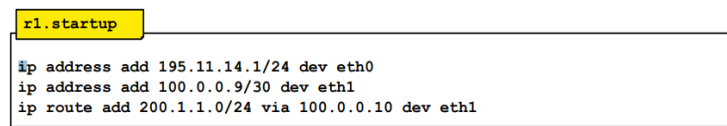


Figure 16

Per vedere gli indirizzi assegnati alle interfacce eseguiamo il comando: ip address.

```
pc1
root@pc1:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
7: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
    link/ether 00:00:00:00:00:01 brd ff:ff:ff:ff:ff:ff
    inet 195.11.14.5/24 scope global eth0
        valid_lft forever preferred_lft forever
```

Figure 17

Per vedere se è presente una rotta di default utilizziamo `route`

```
pc1
root@pc1:/# route

```

Dst	Gateway	Prefsrc	Protocol	Scope	Dev	Table
default	195.11.14.1				eth0	
195.11.14.0/24		195.11.14.5	kernel	link	eth0	
127.0.0.0/8		127.0.0.1	kernel	host	lo	local
127.0.0.1		127.0.0.1	kernel	host	lo	local
127.255.255.255		127.0.0.1	kernel	link	lo	local
195.11.14.5		195.11.14.5	kernel	host	eth0	local
195.11.14.255		195.11.14.5	kernel	link	eth0	local

Figure 18

Se lo utilizziamo invece sul router possiamo vedere la tabella di instradamento. Ora vogliamo vedere il comportamento ARP. In particolare andiamo a vedere prima la tabella ARP senza l'invio di nessun pacchetto:

```
pc3
root@pc3:/# arp -n
root@pc3:/# ping 200.1.1.7
PING 200.1.1.7 (200.1.1.7) 56(84) bytes of data:
64 bytes from 200.1.1.7: icmp_seq=1 ttl=64 time=1.93 ms
64 bytes from 200.1.1.7: icmp_seq=2 ttl=64 time=0.638 ms
--- 200.1.1.7 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.638/1.283/1.929/0.645 ms
root@pc3:/# arp -n

```

Address	Hwtype	Hwaddress	Flags	Mask	Iface
200.1.1.7	ether	00:00:00:00:00:02	C		eth0

Figure 19

Inizialmente il comando non restituirà nulla essendo l'ARP cache vuota, dopo aver eseguito da pc3 un ping a pc2 memorizzerà, l'indirizzo di pc2. Le comunicazioni sono **bidirezionali**, infatti una volta aver ricevuto un **ARP Request** anche il destinatario memorizzerà nell **ARP cache** l'indirizzo del mittente. Con Wireshark possiamo osservare i pacchetti ARP, in particolare **ARP Request** e **ARP Reply**. Notiamo successivamente che alla fine del ping vi è un ulteriore arp request/reply unicast.

Ora ripetiamo lo stesso esperimento su due computer che stavolta, non sono sulla stessa LAN. Questa volta su pc2 verrà memorizzato l'indirizzo del primo Router (r2) che inoltra la sua richiesta. Quando arriva una ARP request a un Router loro la eseguono a loro volta, memorizzandone **mittente** e **destinatario/altro router** (se dovesse raggiungere un'altro router rifarà la stessa cosa).

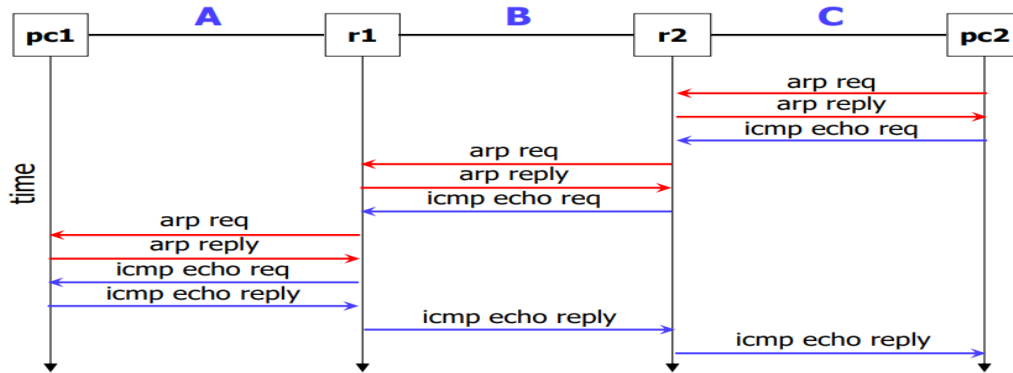


Figure 20

Andiamo ora ad analizzare **Traceroute**

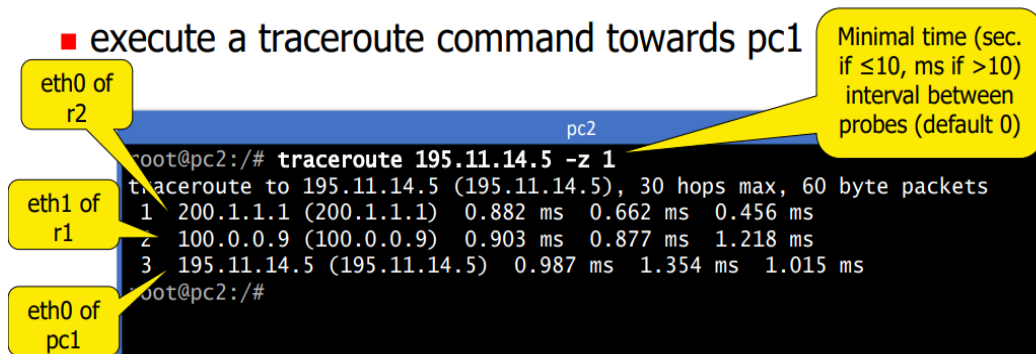


Figure 21

Analizzando wireshark osserviamo i pacchetti con **TTL** basso e i relativi **Time-to-live exceeded** da parte di ICMP. Cliccando sul pacchetto è possibile vedere il TTL del pacchetto in basso a sinistra. In questo caso venivano mandati 3 pacchetti e poi veniva incrementato il **TTL**. Durante il dialogo avvengono degli arp unicast query.

5 Quarto Lab: IPv6 Configurazione di base, ping, traceroute e ICMPv6

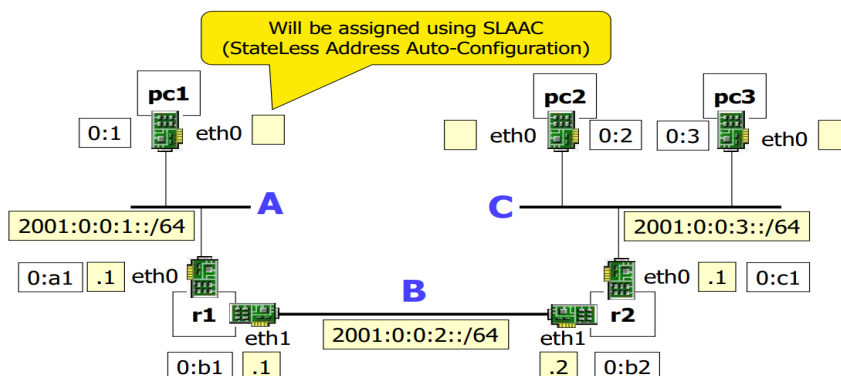


Figure 22

Le differenze rispetto ai lab visti fino ad ora sono poche, in particolare utilizziamo gli indirizzi IPv6 e a ogni pc inseriamo il comando che accetta i router advertisement. Sullo startup non c'è nulla dato che c'è l'**autoconfigurazione stateless**.

```
r1.startup
ip address add 2001:0:0:1::1/64 dev eth0
ip address add 2001:0:0:2::1/64 dev eth1

ip route add 2001:0:0:3::/64 via fe80::200:ff:fe00:b2 dev eth1

chmod o-rw /etc/radvd.conf
systemctl start radvd
```

Figure 23

Vengono messi degli indirizzi IPv6 alle interfacce del router. Viene inoltre aggiunta una riga per far capire al router come raggiungere una LAN che non è direttamente connessa. Il nexthop è un indirizzo link-local. La linea in cui c'è chmod serve a mettere i privilegi corretti per radvd.conf e inizializzare il servizio radvd (router advertisement). Il suo configuration file è nella cartella di r1.

```
radvd.conf
interface eth0
{
    AdvSendAdvert on;
    MinRtrAdvInterval 3;
    MaxRtrAdvInterval 9;
    AdvDefaultLifetime 27;
    prefix 2001:0:0:1::/64 {};
};
```

Figure 24

In questo file andiamo a specificare dove il router manderà gli advertisement (interface eth0), andiamo a mettere in on il fatto che il router manderà gli advertisement, settiamo il massimo e minimo intervallo tra advertisement

consecutivi, il tempo di intervallo di default per la validità del gateway e il prefisso annunciato. Come per IPv4 utilizziamo **ip address** per vedere le interfacce IPv6 di r1.

loopback
IPv4: 127.0.0.1/8
IPv6: ::1/128

eth0
global unicast: 2001:0:0:1::1/64
link-local: fe80::200:ff:fe00:a1/64

eth1
global unicast: 2001:0:0:2::1/64
link-local: fe80::200:ff:fe00:b1/64

```

r1
root@r1:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
11: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 00:00:00:00:00:a1 brd ff:ff:ff:ff:ff:ff
    inet6 2001:0:0:1::1/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::200:ff:fe00:a1/64 scope link
        valid_lft forever preferred_lft forever
12: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 00:00:00:00:00:b1 brd ff:ff:ff:ff:ff:ff
    inet6 2001:0:0:2::1/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::200:ff:fe00:b1/64 scope link
        valid_lft forever preferred_lft forever

```

Figure 25

Questa volta per vedere la tabella di instradamento IPv6 utilizziamo **route6 -f**. Utilizziamo **ip address** sulle macchine per vedere l'indirizzo IPv6 assegnato automaticamente. Utilizziamo **route6 -f** per controllare la presenza di una rotta di default. Ora attacchiamo Wireshark per vedere il comportamento di **ICMPv6**. Quindi su **pc3** ispezioniamo la neighbor cache (ARP cache di IPv6) facciamo un ping a **pc2**, ricontrolliamo la cache e osserviamo i pacchetti che ha catturato Wireshark. Quello che notiamo è che ora **pc3** ha nella neighbor cache l'indirizzo globale IPv6 di **pc2**. Anche qui le comunicazioni sono bidirezionali di conseguenza anche **pc2** avrà memorizzato l'indirizzo globale IPv6 di **pc3**.

WIRESHARK

neighbor solicitation

neighbor advertisement

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	fe80::200:ff:fe00:c1	ff02::1	ICMPv6	110	Router Advertisement from 00:00:00:00:00:c1
2	8.851458135	fe80::200:ff:fe00:c1	ff02::1	ICMPv6	110	Router Advertisement from 00:00:00:00:00:c1
3	9.676431959	2001::3:200:ff:fe00:3	ff02::1:ff00:2	ICMPv6	86	Neighbor Solicitation for 2001::3:200:ff:fe00:2 from 00:00:00:00:00:03
4	9.676594079	2001::3:200:ff:fe00:2	2001::3:200:ff:fe00:3	ICMPv6	86	Neighbor Advertisement 2001::3:200:ff:fe00:2 (rtr, sol, ovr) is at 00:00:00:00:00:02
5	9.676718552	2001::3:200:ff:fe00:3	2001::3:200:ff:fe00:2	ICMPv6	118	Echo (ping) request id=0x0004, seq=1, hop limit=64 (reply in 6)
6	9.676832117	2001::3:200:ff:fe00:2	2001::3:200:ff:fe00:3	ICMPv6	118	Echo (ping) reply id=0x0004, seq=1, hop limit=64 (request in 5)
7	10.733968276	2001::3:200:ff:fe00:3	2001::3:200:ff:fe00:2	ICMPv6	118	Echo (ping) request id=0x0004, seq=2, hop limit=64 (reply in 8)
8	10.734024563	2001::3:200:ff:fe00:2	2001::3:200:ff:fe00:3	ICMPv6	118	Echo (ping) reply id=0x0004, seq=2, hop limit=64 (request in 7)
9	11.774587008	2001::3:200:ff:fe00:3	2001::3:200:ff:fe00:2	ICMPv6	118	Echo (ping) request id=0x0004, seq=3, hop limit=64 (reply in 10)
10	11.774790187	2001::3:200:ff:fe00:2	2001::3:200:ff:fe00:3	ICMPv6	118	Echo (ping) reply id=0x0004, seq=3, hop limit=64 (request in 9)
11	14.734061717	fe80::200:ff:fe00:2	2001::3:200:ff:fe00:3	ICMPv6	86	Neighbor Solicitation for 2001::3:200:ff:fe00:3 from 00:00:00:00:00:02
12	14.734322871	2001::3:200:ff:fe00:3	fe80::200:ff:fe00:2	ICMPv6	78	Neighbor Advertisement 2001::3:200:ff:fe00:3 (rtr, sol)

Frame 1: 110 bytes on wire (880 bits), 110 bytes captured (880 bits) on interface...

Ethernet II, Src: 00:00:00:00:00:c1 (00:00:00:00:00:c1), Dst: IPv6mcast_01 (33:00:00:00:00:01)

Internet Protocol Version 6, Src: fe80::200:ff:fe00:c1, Dst: ff02::1

Internet Control Message Protocol v6

At the end of the ping a unicast neighbor solicitation/advertisement dialogue takes place

Figure 26

Ora dopo aver aggiunto wireshark al dominio di collisione B vogliamo osservare cosa succede facendo un ping da pc2 a pc1. pc2 impara il mac address di r2 su eth0.

```

r1
root@r1:/# ip neigh
fe80::200:ff:fe00:b2 dev eth1 lladdr 00:00:00:00:00:b2 router STALE
fe80::200:ff:fe00:1 dev eth0 lladdr 00:00:00:00:00:01 router STALE
2001::1:200:ff:fe00:1 dev eth0 lladdr 00:00:00:00:00:01 router STALE
fe80::200:ff:fe00:a1 dev eth0 lladdr 00:00:00:00:00:a1 router STALE

r2 (eth1)
root@r2:/# ip neigh
fe80::200:ff:fe00:c1 dev eth0 lladdr 00:00:00:00:00:c1 router STALE
fe80::200:ff:fe00:3 dev eth0 lladdr 00:00:00:00:00:03 router STALE
fe80::200:ff:fe00:b1 dev eth1 lladdr 00:00:00:00:00:b1 router STALE
2001::3:200:ff:fe00:2 dev eth0 lladdr 00:00:00:00:00:02 router STALE
fe80::200:ff:fe00:2 dev eth0 lladdr 00:00:00:00:00:02 router STALE

```

Figure 27

Su wireshark osserviamo un Neighbor Solicitation per capire quale fosse il mac address di r1.

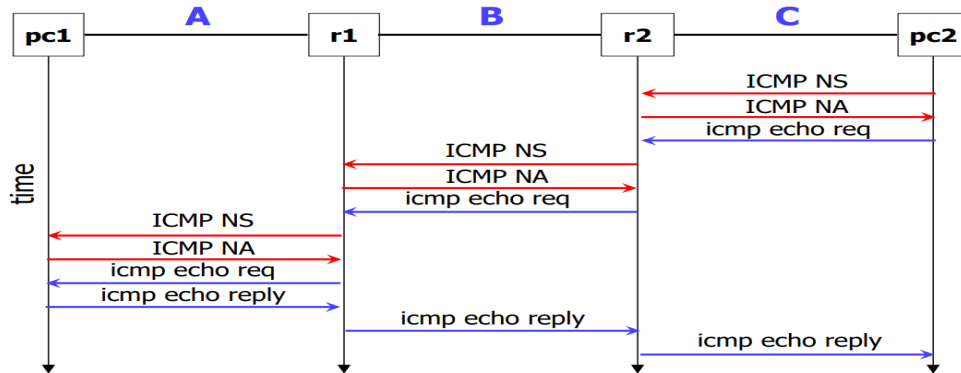


Figure 28

Ora vogliamo osservare il comportamento di ICMPv6 facendo traceroute da pc2 a pc1, per farlo attacchiamo wireshark al dominio di collisione C.

■ execute a traceroute command towards pc1

```

pc2
root@pc2:/# traceroute 2001::1:200:ff:fe00:1 -z 1
traceroute to 2001::1:200:ff:fe00:1 (2001::1:200:ff:fe00:1), 30 hops
max 80 byte packets
 1  2001:0:0:3::1 (2001:0:0:3::1)  0.442 ms  0.580 ms  0.602 ms
 2  2001:0:0:2::1 (2001:0:0:2::1)  0.878 ms  1.175 ms  0.751 ms
 3  2001::1:200:ff:fe00:1 (2001::1:200:ff:fe00:1)  1.078 ms  1.434 ms
    1.708 ms
root@pc2:/#

```

Figure 29

Osserviamo quindi pacchetti UDP e pacchetti ICMP Time-To-Live exceeded

6 Quinto Lab: DNS

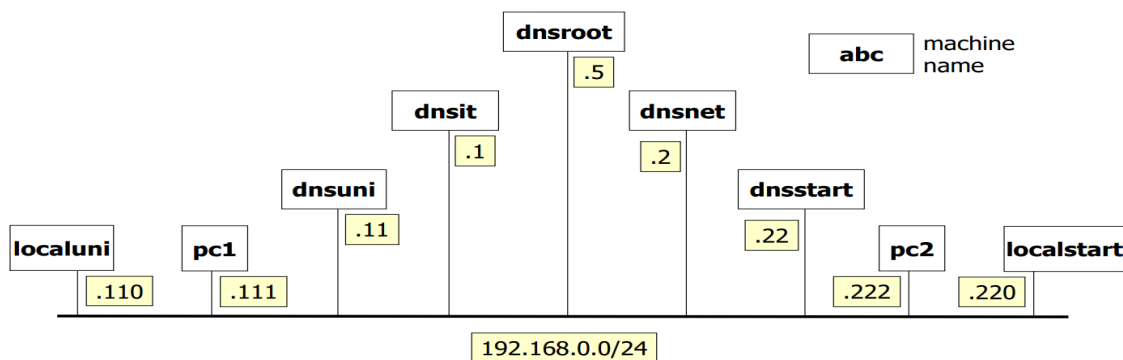


Figure 30

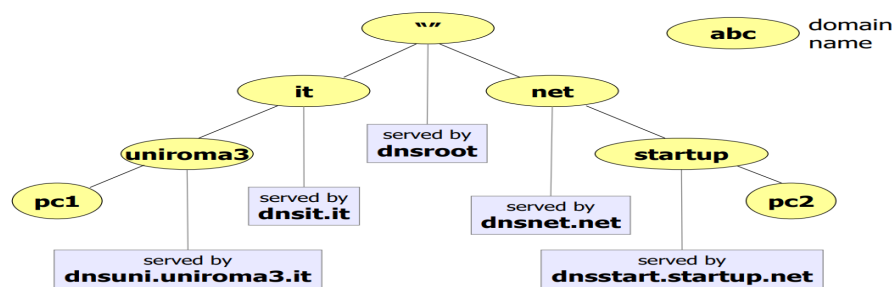


Figure 31

La configurazione dei vari PC consiste sostanzialmente nello specificare il **default name server**



Figure 32

Sui name server invece vengono specificati: associazioni tra zone e name server, informazioni sulla radice dei name server, associazioni tra nomi e indirizzi IP e l'autorizzazione a risolvere query ricorsive.

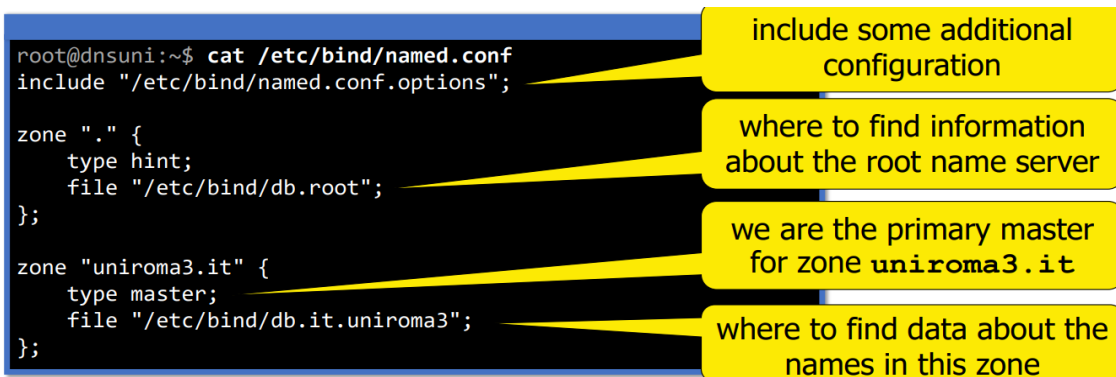


Figure 33

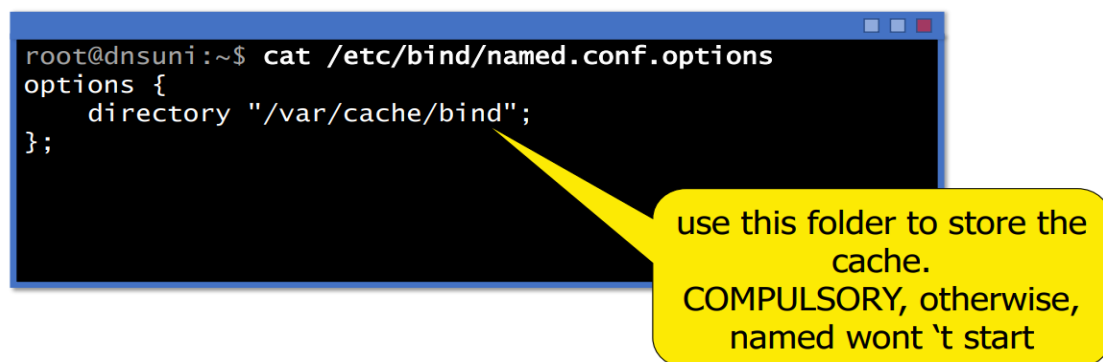


Figure 34

Informazioni sulla radice dei name server

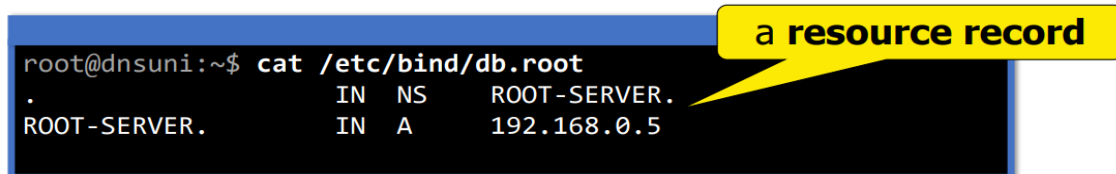


Figure 35

Informazioni come il TTL e altre

```

root@dnsuni:~$ cat /etc/bind/db.it.uniroma3
$TTL      60000
@          IN      SOA      dnsuni.uniroma3.it.
root.dnsuni.uniroma3.it. (
                        2024120401 ; serial
                        28 ; refresh
                        14 ; retry
                        3600000 ; expire
                        0 ; negative cache ttl
                        )

```

makes sense for master/slave configuration

Figure 36

La chiocciola in alto a sinistra sta ad indicare che questo record si riferisce all'origine attuale, Tutti i nomi di dominio in questo file di dati che non sono completamente qualificati (non terminano con un '.') sono relativi all'origine. L'**origine** è il nome di dominio nella dichiarazione della zona del file di configurazione del server. La riga che riporta **IN SOA dnsuni.uniroma3.it.** sta ad indicare il server master primario per questa zona (dnsuni.uniroma3.it), non bisogna dimenticare il punto finale altrimenti il nome dell'origine (uniroma3.it) verrebbe aggiunto. La riga che riporta **root.dnsuni.uniroma3.it** è l'indirizzo email della persona responsabile per quella zona (il primo . va scambiato con una @ root@dnsuni.uniroma3.it). Le righe evidenziate nella foto servono per le configurazioni dei server master e slave. **Serial number** indica quanto recente è l'informazione (è una data YYYYMMDDNN). **Refresh interval** indica allo slave quando spesso controllare i dati della zona se sono aggiornati. **Interval** è l'intervallo tra tentativi consecutivi per ricontattare il master. **Slave expire time** se lo slave fallisce a contattare il master per questa quantità di tempo, considera i dati della zona troppo vecchi e smette di dare informazioni su quest ultima. **TTL per risposte negativi**.

```

root@dnsuni:~$ cat /etc/bind/db.it.uniroma3
$TTL      60000
@          IN      SOA      dnsuni.uniroma3.it.
root.dnsuni.uniroma3.it. (
                        2024120401 ; serial
                        28 ; refresh
                        14 ; retry
                        3600000 ; expire
                        0 ; negative cache ttl
                        )
@          IN      NS       dnsuni.uniroma3.it.
dnsuni.uniroma3.it. IN      A       192.168.0.11
pc1.uniroma3.it.   IN      A       192.168.0.111

```

record type NS (name server)

the authoritative name server for this zone (uniroma3.it) is dnsuni.uniroma3.it

Figure 37

Sono segnate le macchine presenti in questa zona.

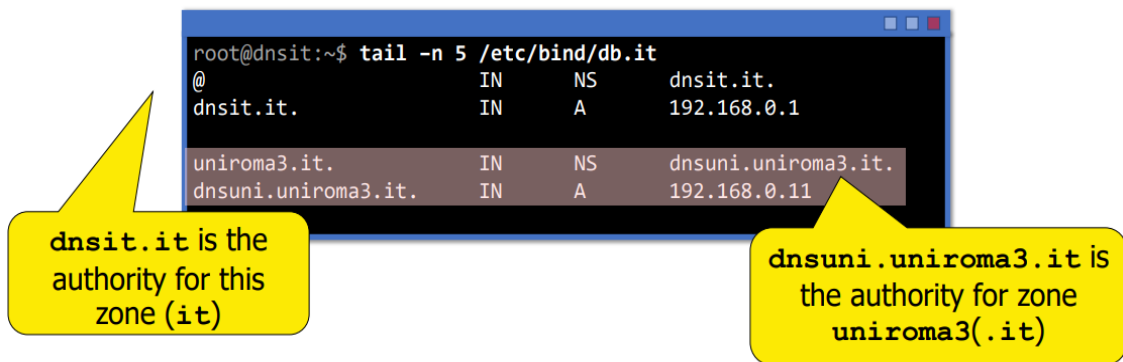


Figure 38

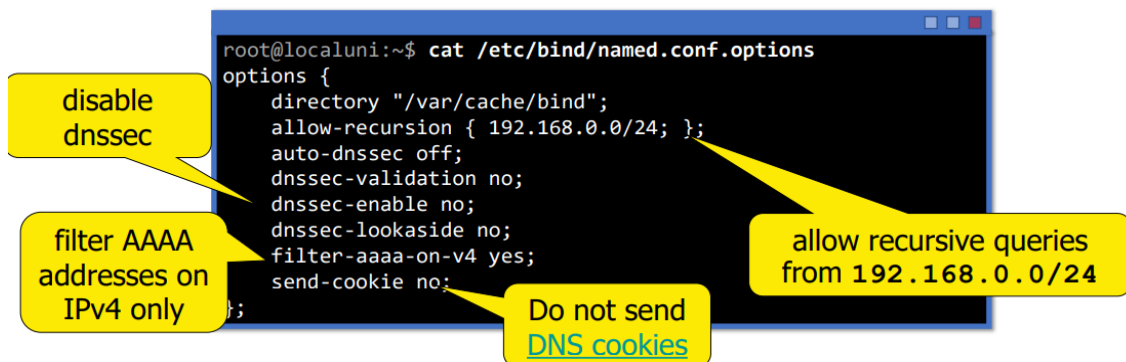


Figure 39

Ora iniziamo il lab, aggiungiamo wireshark al dominio di collisione a e facciamo un ping da pc1 a pc2.

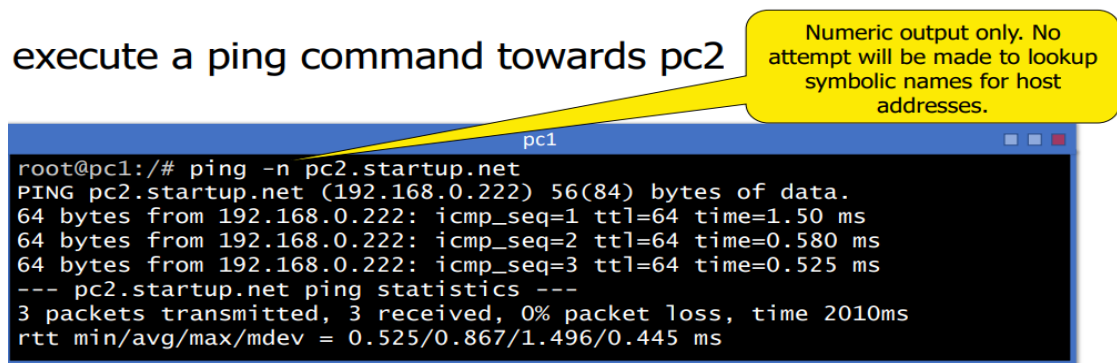


Figure 40

Applicando su wireshark il filtro per vedere sono i pacchetti DNS, possiamo notare tutte le varie query

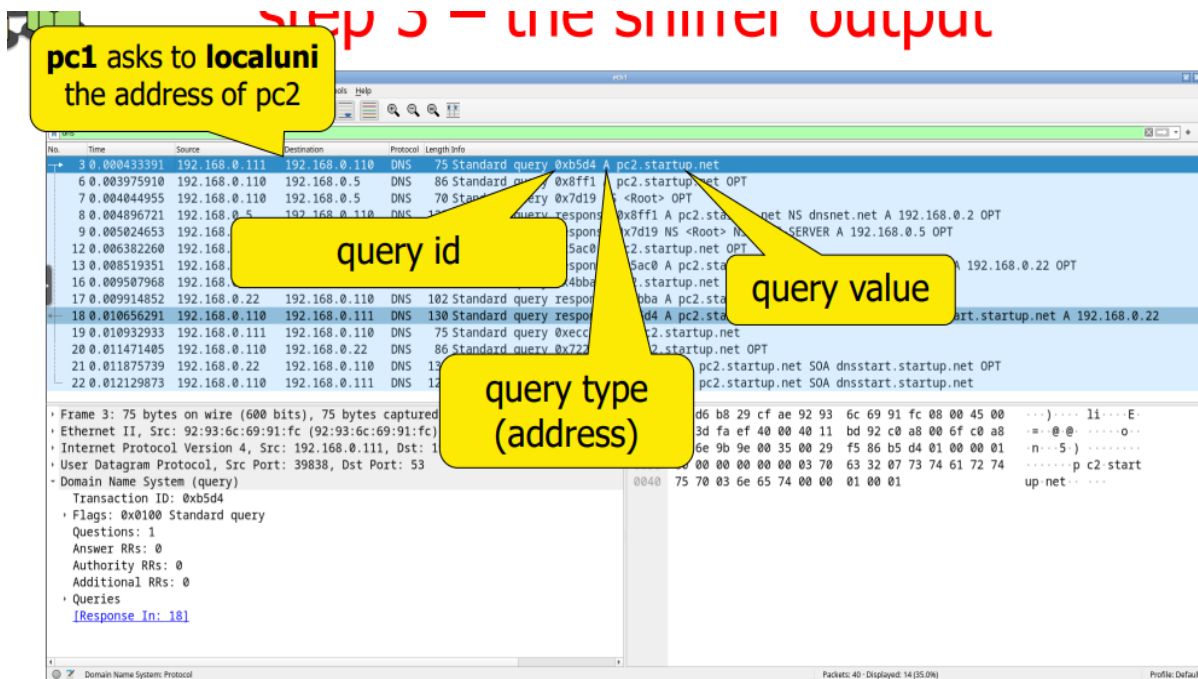


Figure 41

Con `< Root >` possiamo vedere le richieste root name server. Quindi quello che succede è **pc1** chiede a **localuni** l'indirizzo di **pc2** il quale a sua volta chiede a **dnsroot** qual è il name server per il dominio **net**, riceve risposta **192.168.0.2**. Successivamente **localuni** chiede a **dnsnet** chi è il name server per il dominio **startup.net**. Riceve risposta, l'indirizzo di **dnsstart.startup.net** è **192.168.0.22**. Successivamente **localuni** chiede a **dnsstart** qual'è l'indirizzo di **pc2.startup.net**, riceve risposta, l'indirizzo di **pc2.startup.net** è **192.168.0.222**. **Localuni** riporta a **pc1** l'indirizzo di **pc2.startup.net**. Successivamente **pc1** chiede a **localuni** per i record AAAA di **pc2.startup.net**, ora **localuni** chiede direttamente a **dnsstart** (perchè già lo conosce) per i record AAAA di **pc2.startup.net**, **dnsstart** riporta un SOA record nel quale indica che non ha nessun record AAAA per **pc2.startup.net**, **localuni** restituisce la risposta di **dnsstart** a **pc1**. (Tutto questo è visualizzabile su wireshark e graficamente sulle slide). Ora se venisse ripetuto l'esperimento la cache del name server ridurrebbe il traffico che c'è stato per la prima volta. **Per pulire la cache:**

```
localuni
root@localuni:/# rndc flush
```

- **rndc controls the operation of a name server**
- **the flush command cleans up caches**
 - a new client query triggers the complete sequence of iterative queries

Figure 42

Ora eseguiamo un esperimento mandando un ping verso un dispositivo inesistente, quello che succederà è che a causa della pulizia della cache tutte le query iterative verranno rifatte, fino a quando si accorgeranno che il dominio richiesto non esiste. Quando la query ritornerà failed **pc1** riprova una seconda volta con il percorso di ricerca del dominio configurato nel suo `/etc/resolv.conf`:

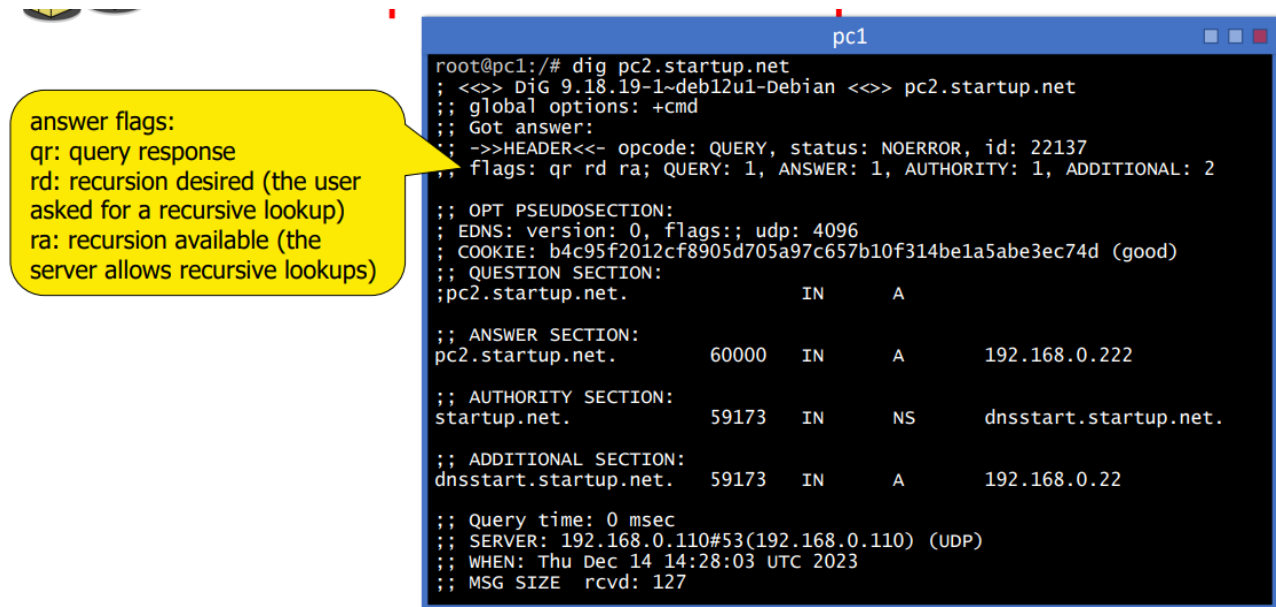
```
nameserver 192.168.0.11
search uniroma3.it
```

Se ripetiamo ancora una volta il ping verso un dispositivo inesistente, la negative cache del name server avrà memorizzato l'esito negativo quindi restituirà subito l'esito e **pc1** riproverà con il percorso di ricerca del dominio

configurato nel suo `/etc/resolv.conf`.

6.1 Il comando `dig`

I resource record possono essere cercati utilizzando il comando `dig`.



```
root@pc1:/# dig pc2.startup.net
; <<> DiG 9.18.19-1~deb12u1-Debian <<> pc2.startup.net
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 22137
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; COOKIE: b4c95f2012cf8905d705a97c657b10f314be1a5abe3ec74d (good)
;; QUESTION SECTION:
;pc2.startup.net.                IN      A

;; ANSWER SECTION:
pc2.startup.net.                60000   IN      A      192.168.0.222

;; AUTHORITY SECTION:
startup.net.                    59173   IN      NS      dnsstart.startup.net.

;; ADDITIONAL SECTION:
dnsstart.startup.net.          59173   IN      A      192.168.0.22

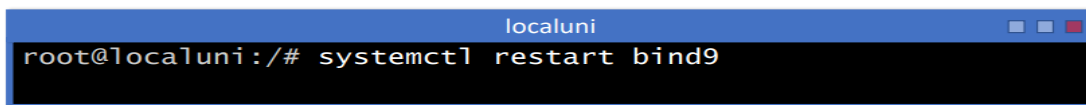
;; Query time: 0 msec
;; SERVER: 192.168.0.110#53(192.168.0.110) (UDP)
;; WHEN: Thu Dec 14 14:28:03 UTC 2023
;; MSG SIZE rcvd: 127
```

answer flags:
qr: query response
rd: recursion desired (the user asked for a recursive lookup)
ra: recursion available (the server allows recursive lookups)

Figure 43

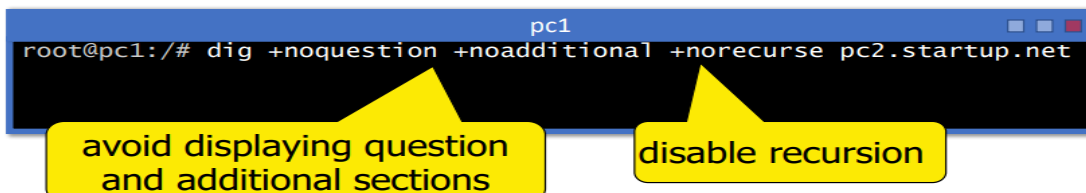
Le sezioni corrispondono a quelle contenute nei pacchetti DNS. Un messaggio DNS non contiene mai più di una sezione di domanda. Nella sezione delle risposte invece potrebbero essere più di uno. Quei numeri ad es. (60 000) sono i TTL dei resource record e possono decrementare o rimanere costanti se il record è memorizzato sul name server.

restart bind on the name server



```
root@localuni:/# systemctl restart bind9
```

perform an iterative query using `dig`



```
root@pc1:/# dig +noquestion +noadditional +norecurse pc2.startup.net
```

avoid displaying question and additional sections

disable recursion

Figure 44

Il server risponde specificando il name server autorevole da contattare per ottenere le informazioni desiderate.

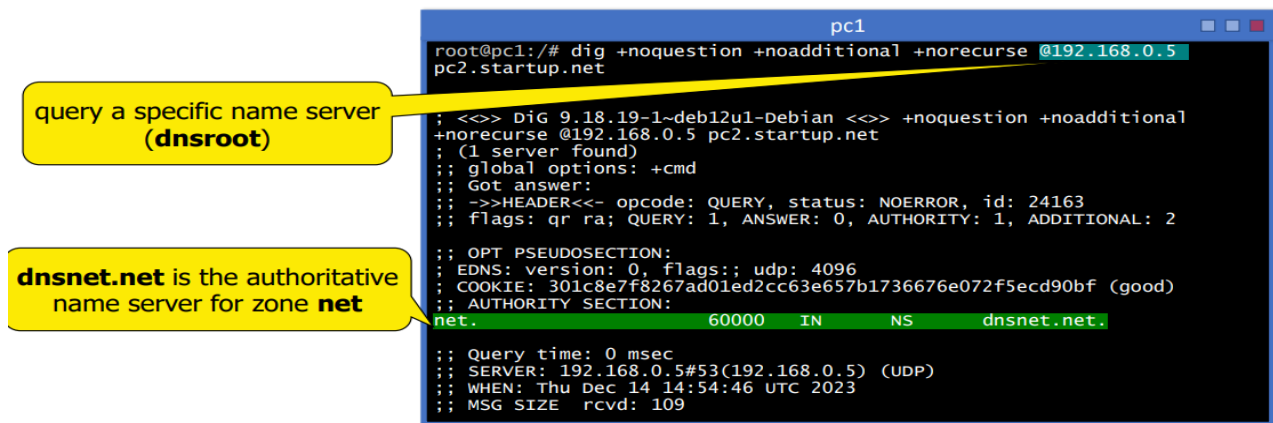


Figure 45

Scendendo sempre di più con i name server arriveremo a **dnsstart.startup.net** il quale ci restituirà come risposta l'indirizzo di **pc2.startup.net**

7 Sesto Lab: web server e browser

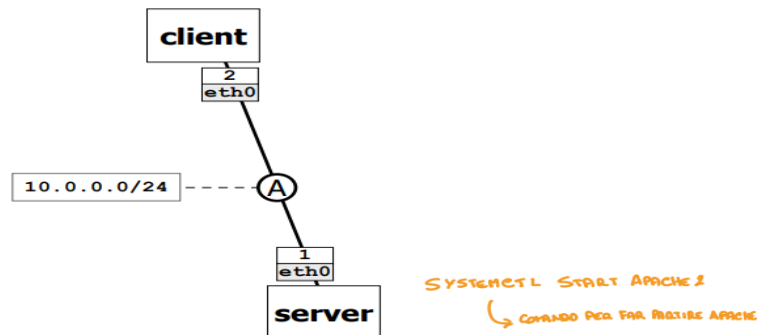


Figure 46

Il server esegue **apache2**. Il client utilizza un web browser text-based (**links**) per controllare le operazioni del server. **systemctl start apache2** per eseguire apache. Successivamente il client esegue **links http://10.0.0.1** per inizializzare il web browser. Successivamente attacchiamo **wireshark** per vedere cosa succede. Catturiamo 13 pacchetti che sono gli stessi che avevamo visto per HTTP.

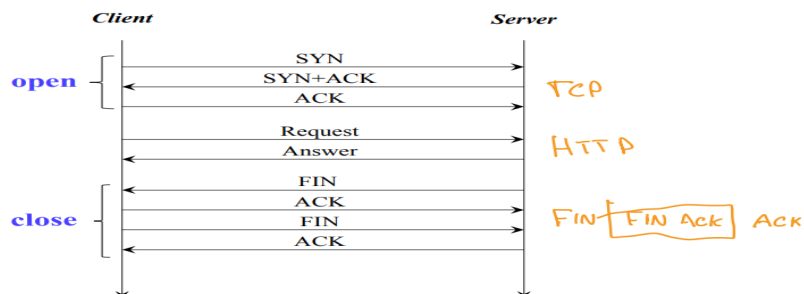


Figure 47

(Apri wireshark per seguire meglio)

Il primo pacchetto è un **ARP Request** il client cerca il MAC address del server. Il secondo è un **ARP Reply** il server gli fornisce l'indirizzo MAC. Ora inizia il **three-way-handshake**. Nel primo pacchetto il client propone il sequence number e una grandezza massima del segmento (1460byte). Il secondo pacchetto è un **syn ack** in cui si conferma la grandezza massima del segmento, ora il server propone il suo sequence number iniziale e conferma quello proposto dal client. L'ultimo pacchetto è un **ACK** in cui il client conferma il sequence number proposto dal server. Il sesto pacchetto è un pacchetto **HTTP GET** a cui segue il settimo pacchetto in cui viene confermato il GET da **TCP**. Nell'ottavo pacchetto c'è la **risorsa richiesta**. Nel nono **TCP** conferma con un **ACK** la risorsa. Dopo il decimo, finisce la conversazione e quindi **FIN, ACK, FIN, ACK**.

